

DriveShare

Peer-to-Peer Car Rental Platform

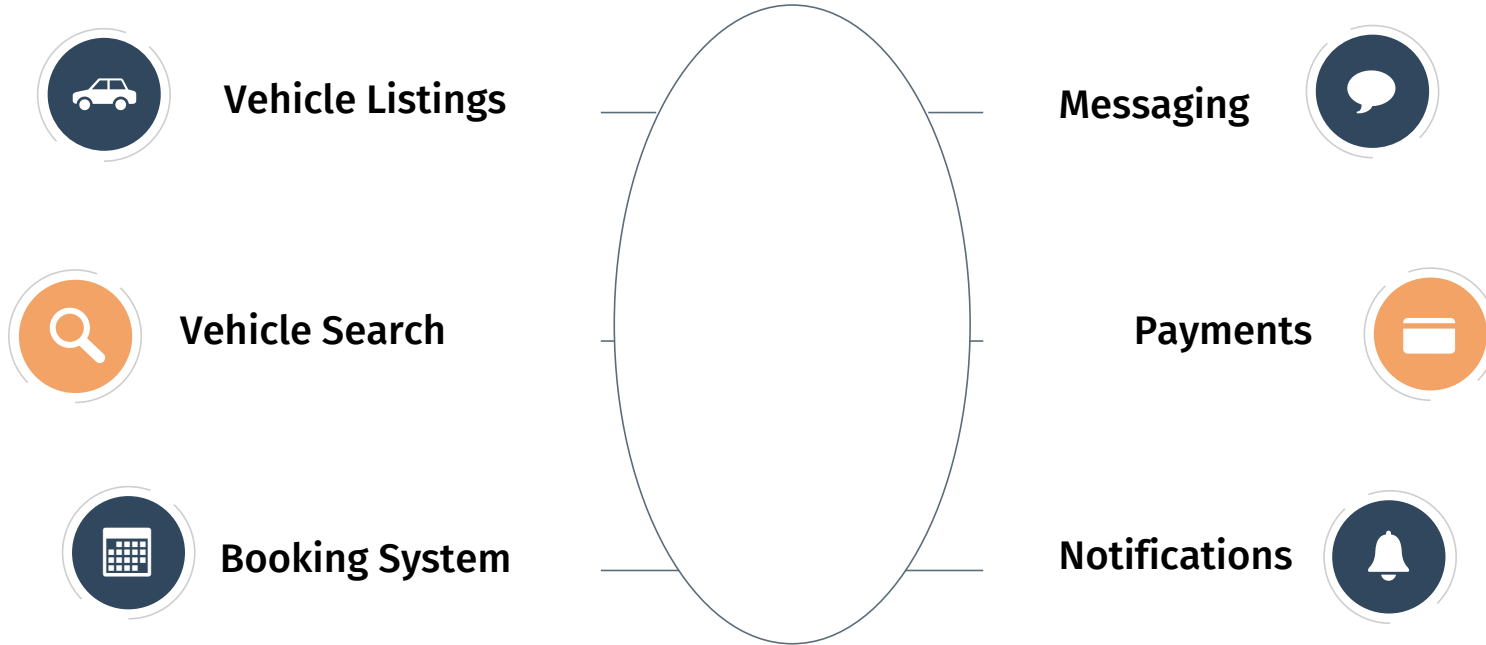
Reem Jarrar

Term Project

CIS 566 – Software Architecture
and Design patterns



Project Objectives



Live Demo



Register

Name	Sarah
Email	owners@test.com
Password	***
Security Question 1	Where did you grow up?
Answer 1	Detroit
Security Question 2	Name of first pet?
Answer 2	Cinnamon
Security Question 3	Favorite Food?
Answer 3	Pizza
Role	owner

[Create Account](#)[Back to Login](#)

DriveShare Login

Email

owner@test.com

Password

Login

Register New User

Forgot Password

Owner Dashboard

[Logout](#)[Add Car](#) [My Cars](#) [Messages](#) [Notifications](#)

Model	Toyota Camry
Year	2022
Mileage	25000 miles
Location	Detroit
Price Per Day	90
Available From YYYY-MM-DD	2026-01-01
Available To YYYY-MM-DD	2026-06-30
Description	New Car

[Add Listing](#)

Owner Dashboard

[Logout](#)[Add Car](#) [My Cars](#) [Messages](#) [Notifications](#)

Id	Model	Location	Price	Dates
1	Toyota Camry	Detroit	\$90.00	2026-01-01 to 2026-06-30

New Price [Update Selected Car Price](#)

Renter Dashboard

[Logout](#)[Search & Book](#) [Payment](#) [Messages](#) [Notifications](#)Location Start YYYY-MM-DD End YYYY-MM-DD Max Price

Id	Model	Year	Location	Price	Dates
1	Toyota Camry	2022	Detroit	\$90.00	2026-01-01 to 2026-06-30

Watch Target Price

Renter Dashboard

[Logout](#)[Search & Book](#) [Payment](#) [Messages](#) [Notifications](#)Location Start YYYY-MM-DD End YYYY-MM-DD Max Price

Id	Model	Year	Location	Price	Dates
1	Toyota Camry	2022	Detroit	\$90.00	2026-01-01 to 2026-06-30

 Booking Confirmed

 Booking #1 created. Total amount: \$16290.00

[Book Selected Car](#) [View My Rentals](#) [View My Profile](#) [View My Bookings](#) [View My Reviews](#) [View My Settings](#)

Renter Dashboard

[Logout](#)[Search & Book](#) [Payment](#) [Messages](#) [Notifications](#)Location Start YYYY-MM-DD End YYYY-MM-DD Max Price

Id	Model	Year	Location	Price	Dates
1	Toyota Camry	2022	Detroit	\$90.00	2026-01-01 to 2026-06-30

 Watchlist ×

 Car added to watchlist.

Watch Target Price

Owner Dashboard

[Logout](#)[Add Car](#) [My Cars](#) [Messages](#) [Notifications](#)

Id	Model	Location	Price	Dates
1	Toyota Camry	Detroit	\$90.00	2026-01-01 to 2026-06-30

 Success

 Price updated. Matching watchers were notified.

OK

New Price [Update Selected Car Price](#)

Renter Dashboard

Logout

Search & Book Payment Messages Notifications

- Good news! Car #1 price dropped to \$50.00, meeting your watchlist target.

Renter Dashboard

Logout

Search & Book Payment Messages Notifications

From Reem to John Owner: Is the pickup location still available. I'm interested in renting your Toyota Camry

Receiver User ID Message

Send

Owner Dashboard

Logout

Add Car My Cars Messages Notifications

From Reem to John Owner: Is the pickup location still available. I'm interested in renting your Toyota Camry

Receiver User ID Message

Password Recovery

Email

owners@test.com

Load Security Questions

Q1: Where did you grow up?

Q2: Name of first pet?

Q3: Favorite Food?

New Password

Reset Password

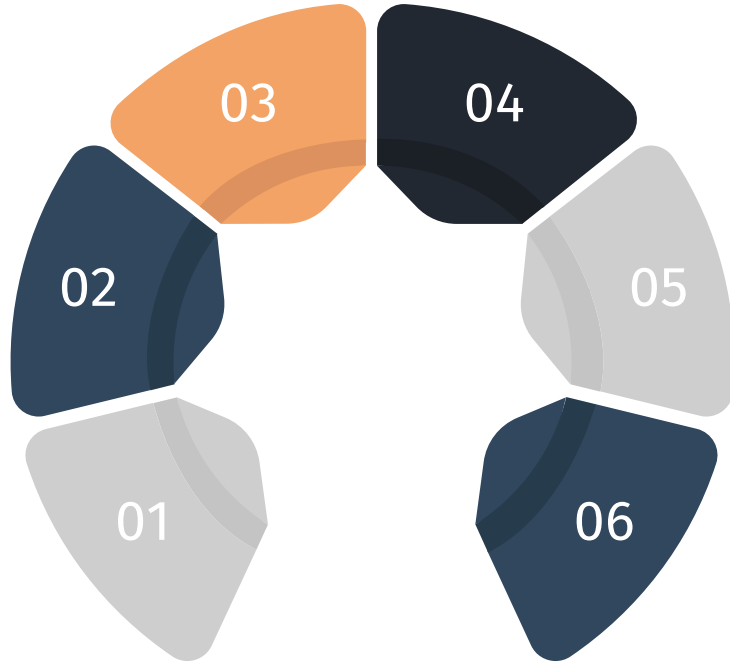
Back to Login

System Architecture

Singleton

Builder

Observer



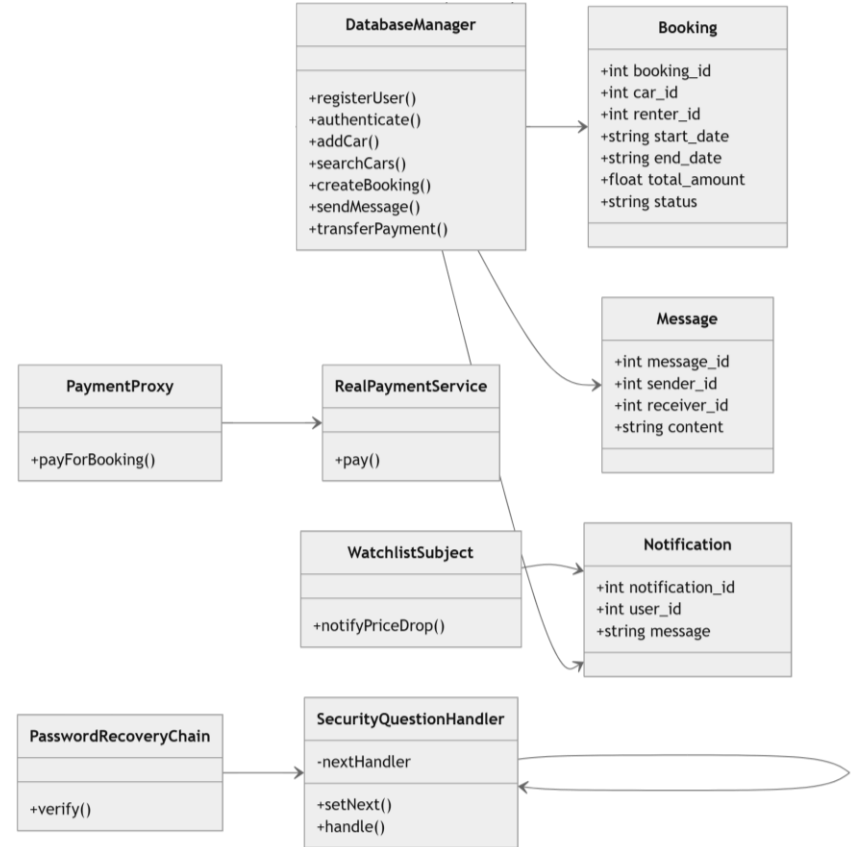
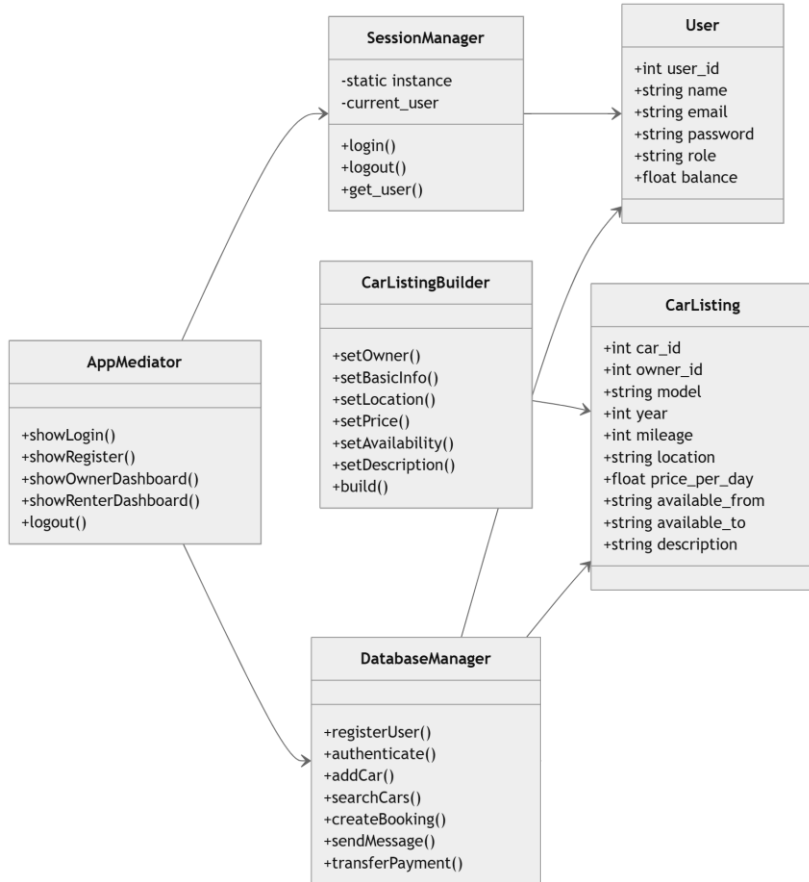
Mediator

Proxy

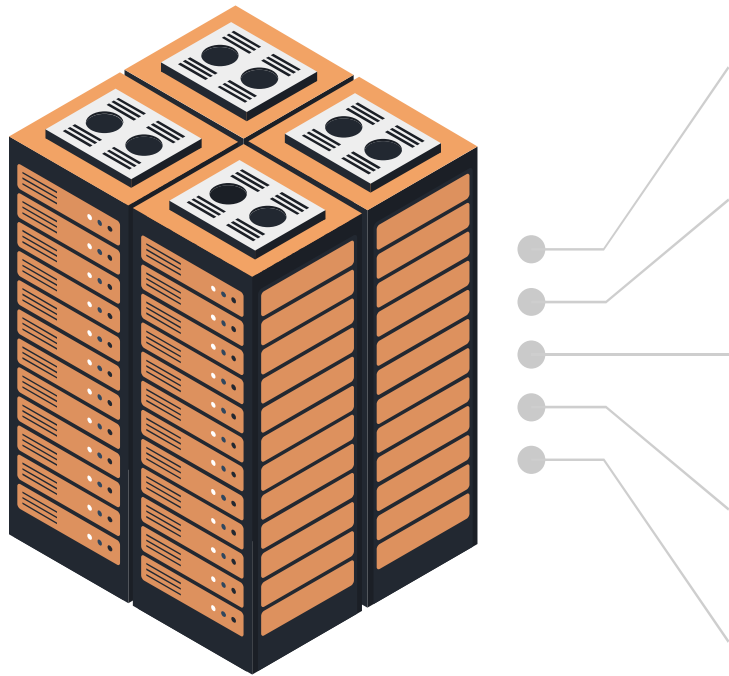
Chain of
Responsibility

Tkinter GUI → Application Layer → Database

UML Diagram



Database Design



1

Users

2

Cars

3

Bookings

4

Watchlists

5

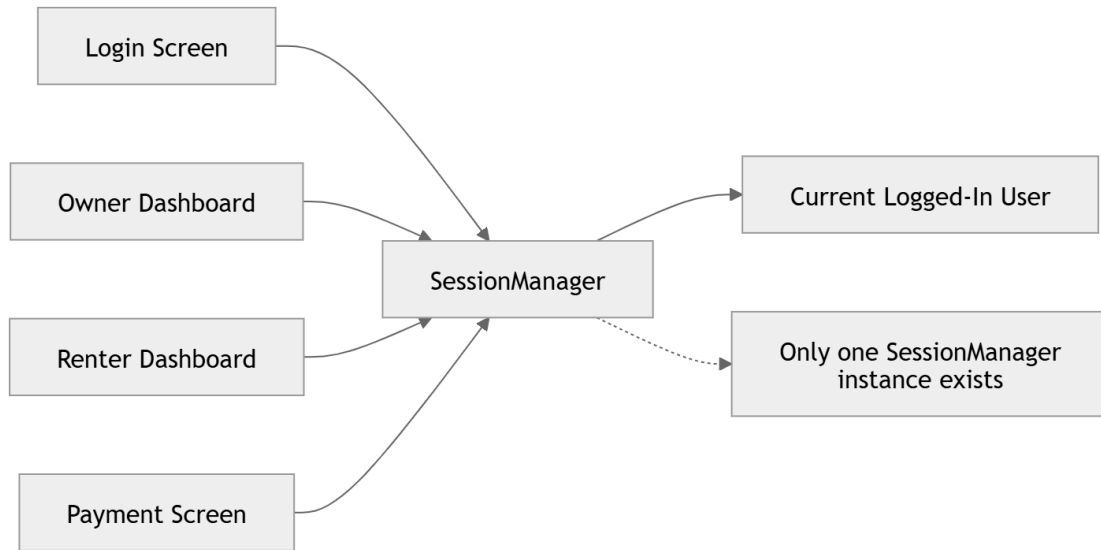
Messages

6

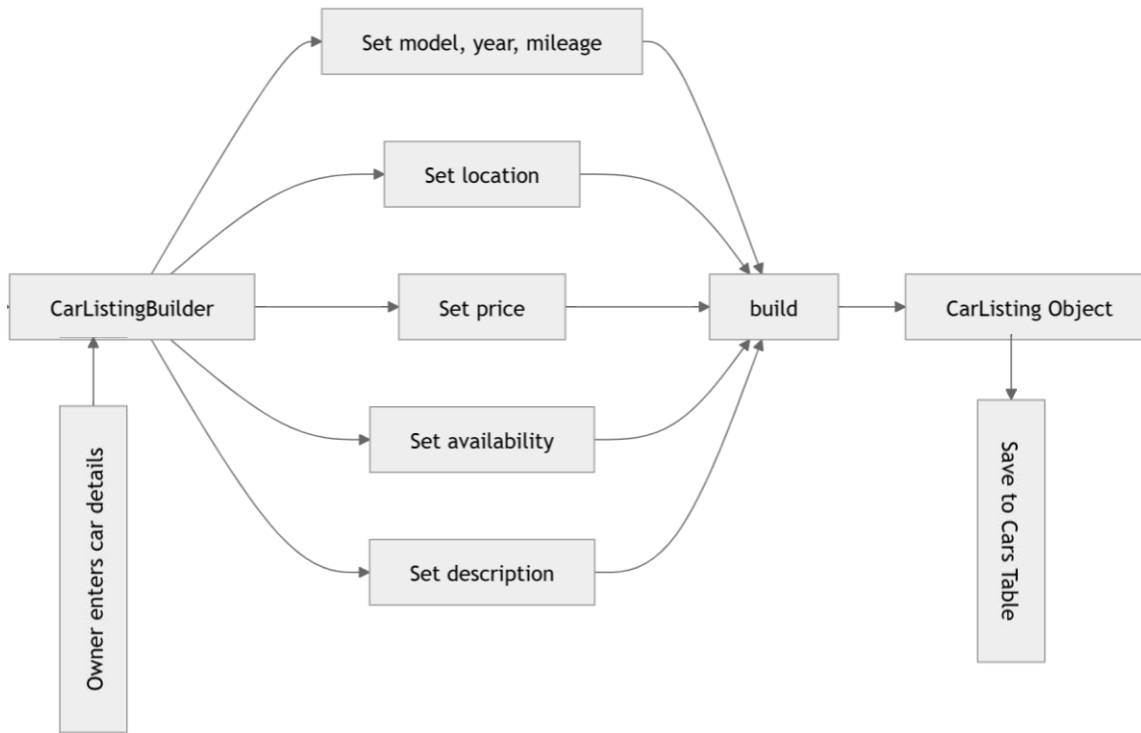
Notifications

Singleton Pattern

```
class SessionManager:  
    _instance = None
```



Builder Pattern

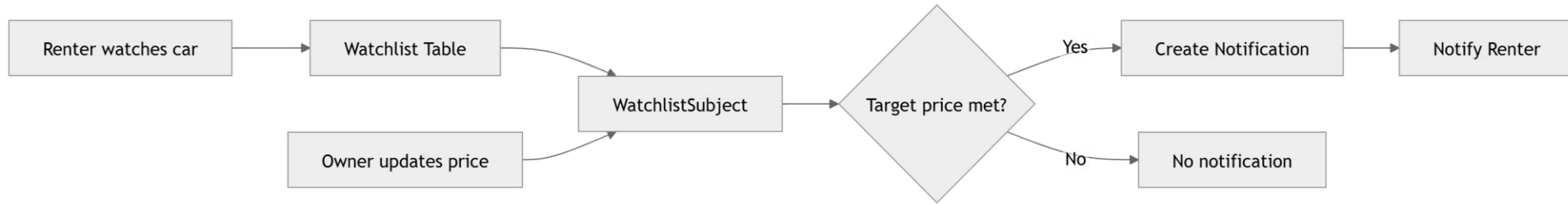


```
# Builder Pattern: creates a car
listing step by step
builder = CarListingBuilder()

car = (
    builder
    .set_owner(user["id"])
    .set_basic_info(model, year,
mileage)
    .set_location(location)
    .set_price(price_per_day)
    .set_availability(available_f
rom, available_to)
    .set_description(description)
    .build()
)

db.add_car(car)
```

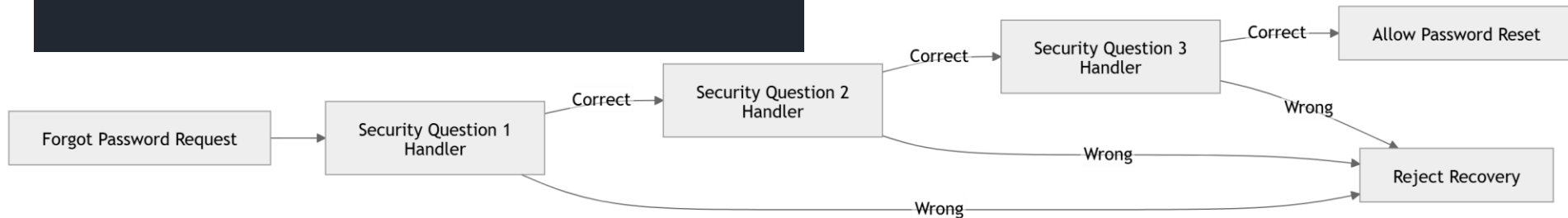
Observer Pattern



```
# Chain of Responsibility: password
recovery
chain = PasswordRecoveryChain(user_record)

if chain.verify(answer1, answer2, answer3):
    db.update_password(user_record["id"],
new_password)
    print("Password reset successful.")
else:
    print("One or more security answers are
incorrect.")
```

Chain of Responsibility



```
class PaymentProxy:
    def pay_for_booking(self, booking_id):
        booking = db.get_booking(booking_id)

        if not booking:
            return False, "Booking not found."

        if booking["paid"]:
            return False, "Booking is already
paid."

        return
self.real_payment_service.pay(booking)
```

Proxy Pattern for Payment

Booking Conflict Prevention

Booking A

|-----May 1-----May 5-----|

Booking B

 |-----May 3-----May 7-----|

✕ Conflict

```
def has_overlapping_booking(car_id,
                             start_date, end_date):
    query = """
        SELECT COUNT(*) FROM bookings
        WHERE car_id = ?
        AND status = 'confirmed'
        AND start_date <= ?
        AND end_date >= ?
    """

    cursor.execute(query, (car_id,
                           end_date, start_date))
    return cursor.fetchone()[0] > 0
```


Conclusion

